



Curriculum vitae

Personal details

Name	Aart den Braber
Email	cv@prosumfrontend.nl
LinkedIn	https://linkedin.com/in/aartdenbraber
Website	https://aartdenbraber.nl

Nice to meet you!

I'm a fullstack developer with a passion for learning new things (see also the courses I've followed) and an eye for detail, especially regarding TypeScript and testing. Next to that, I'm a certified UX-researcher, so my focus is always on the end-user. This may mean that I'm able to figure out the best solution together with a UX designer, but might also mean that you can skip hiring a dedicated one. However, my focus is really on creating applications and writing code to achieve that.

Stack (unsorted): Angular 4-14 · Ionic2 · StencilJS · React · Redux · RxJS · NgRx · Node · TypeORM · NestJS · React · native JavaScript (ES6/2020, maar ook verouderde versies) · jQuery · TypeScript · NPM · yarn · HTML(5) · microdata · Sass · Java · Python · Flask RestX · PHP · Docker · Figma · Balsamiq · AWS (Elastic Beanstalk & S3) · ServiceNow dev · MySQL · PostgreSQL · MongoDB · Passport.js · Gitlab · Github · Cloud Foundry · Kubernetes · Source Tree · testing (TDD · Selenium · Protractor · Jasmine · Jest · Cypress · Spectator · Unit, integration & e2e) · JSDocs/JavaDocs/PyDocs · Prettier · TS/ESLint · OpenAPI 3 · Swagger · FTP · Wordpress · JSON · YAML · XML · Json Web Tokens (JWT) · Redux · RxJS · Express · pgAdmin · Socket.io · webcomponents · StencilJS · BS/Tailwind with PurgeCSS · ...

Methodologies: CI / CD · Git · Git flow · Github flow · designing and implementing REST API's · UX design · Agile / Scrum · testing · Typesafety · Accessibility · Flow design · BEM · ...

Also, I can say anything I like, but maybe my previous colleagues and managers can do a better job of saying what kind of person and developer I am. For their recommendations, please check [LinkedIn.com/in/aartdenbraber](https://linkedin.com/in/aartdenbraber) or the last pages of this CV.

Experience

Feb 2024 - Present

Full stack engineer

DUO (freelance), Den Haag

For my second assignment within DUO (the Dutch Education Executive Agency), I worked on two major domains within DUO's legal status assignment system: **HOST** (Higher Education) and **MBOST** (Vocational Education). The project aimed to modernize both frontend and backend systems and improve maintainability, functionality, and user experience.

What we did as a team:

- Extended and restructured the Angular frontend for HOST, enabling access for both business users and DUO staff (Angular, Jasmine, Spectator);
- Designed and implemented backend REST APIs and integrated them with the frontend (Java, JUnit, Spring Security);
- Created extensive documentation for onboarding, maintainability, and knowledge sharing.

My personal contributions:

- Significantly improved maintainability: reduced complex setup from 10+ manual steps to a streamlined, one-time configuration (Java, Node, Angular);
- For HOST, applied a lift-and-shift approach due to strict business continuity needs — minimal UX improvements possible, but applied optimizations where feasible;
- For MBOST, actively collaborated with the UX designer and API design team to improve usability and align functionality with end-user needs (UX, REST, YAML);
- Refactored and clarified backend processes, including database population, inter-module integrations, and message handling via ActiveMQ;
- Led the greenfield development of the MBOST platform for both internal and business-facing users (Angular, Java, Spring Boot, YAML class generator, Liquibase, Hibernate);
- Mentored a junior fullstack developer and a junior tester, focusing on writing maintainable, testable code and understanding architectural decisions.

Stack (not ordered): Java, Spring, Spring Boot, REST API design, Liquibase, Hibernate (SQL), ActiveMQ, Angular, TypeScript, Node, YAML, GitLab, Jenkins, Jira, CI/CD

Used specializations and methodologies: Agile / Scrum, flow design, test-driven development, UX research, accessibility, Git

Mar 2023 - Feb 2024

Full stack engineer

DUO (freelance), Den Haag

DUO (the Dutch Education Executive Agency) aimed to adopt a case-centric approach for handling documents. At the start of the project, there was a minimal viable product (MVP) in place, but it was not sufficient for a full rollout. Together with the team, I contributed to developing and launching a complete application for managing cases and enabling end-users to efficiently process and link documents to cases.

What we did:

- Updated the existing frontend and rewrote component tests using Angular, NgRx, and Cypress;
- Extended backend functionality to support case-document linking. Planned and executed releases in collaboration with the team;
- Created extensive documentation for maintainability and onboarding.

What I did:

- Refactored and improved existing structures; identified backend weaknesses and proposed solutions;
- Worked closely with the UX/UI designer to align implementation with user needs;
- Actively collaborated with stakeholders to translate their needs into concrete technical solutions.

Stack (not ordered): TypeScript, Angular, NgRx, JavaScript, Java, Jira, GitLab, Jenkins, CI/CD, Cypress (e2e & component), Jest

Used specializations and methodologies: Agile / Scrum, Flow Design, Git, Accessibility, UX Research

Nov 2021 - Mar 2023

Software Engineer

Knab (freelance), Amsterdam

At Knab my team and I have been working to make the transfer of Aegon customers to Knab as smooth as possible. These customers were being transferred because Aegon wanted Knab (where Aegon was a main shareholder) to take over investments and retirement accounts.

What I did:

- Create an Angular application to make for a smooth landing of the customers that have either investments or retirement accounts (using a.o. Angular, PrimeNg, Inbenta);
- Using AWS Lambda (later we switched to AWS ECS) we created a backend-for-frontend to work safely with the very sensitive data we retrieved from the Knab API's (using a.o. NodeTS, Express, DynamoDB, OAuth, jest);
- Wrote loads of tests (80-96% cov.) in React, Angular and Node to make sure the quality is assured (using a.o. jest, Jasmine, Spectator, Wallaby, Mosh's technique). Next to that we implemented NewRelic to keep an automated eye on the apps;
- Used React to create a frontend which is used to login using OAuth, but also allows for changing a password, creating an account, sending/receiving an OTP-code using SMS or email and such (using a.o. React, jest, OAuth);
- Worked towards an MVP, so we had loads of collaboration with different teams to achieve all kinds of integrations;

The result:

A very maintainable application which we created in a multi-disciplinary team with 3 frontenders and a backender. We're currently working towards the MVP, which we hope to deliver in february.

Stack (not ordered): TypeScript, Angular, React, NgRx, NgRx Data, AWS Lambda, AWS ECS, RxJS, NodeTS, Express, Axios, PrimeNg, JavaScript, yarn, GitKraken, Jira, BitBucket, CI / CD, Prettier, ESLint, HTML(5), (S)CSS(3), Cypress, Jasmine, Jest, Wallaby

Used specializations and methodologies: Agile / Scrum, flow design, Git, Accessibility, UX research

• **Lead front-end / UX design**

Politie Nederland (via Ordina), Odijk

In this second project for the Dutch Police, my team (the same Ordina High Performance Team as the previous project) was responsible for making several EBO's maintainable and manageable. EBO's are internal applications that were made by very small teams, but outgrew their original size and thus needed to be setup more professionally.

What I did:

- Talked a lot to the users and original creators of the apps in order to create the best possible apps;
- Gave a lot of feedback on test-techniques and code-standards;
- I implemented loads of things that make the life of a simple developer better (auto-fix ESLINT, good testing standards, good CI/CD);
- Developed multiple tools and made packages for them in order to make them easily reusable.

The result:

An app with which users can very quickly see their requests. Because I made sure to involve the team from the start by taking them to my talks with the users of the app (using the fish-bowl method), the whole team was able to help define and implement the users' wishes. I also worked on an app that allowed officers to quickly see where camera's are located (Camera in Beeld).

Stack (not ordered): JavaScript, NPM, yarn, Source Tree, Jira, BitBucket, CI / CD, Prettier, HTML(5), (S)CSS(3), Docker, Cloud Foundry, Python, Node, Cypress, AlpineJS, ESLint, Node,

Used specializations and methodologies: Agile / Scrum, flow design, Git, Accessibility, UX design, usertests

Apr 2020 - Apr 2021

Lead front-end / UX design

Politie Nederland (via Ordina), Odijk

At the Dutch Police, my team (an Ordina High Performance Team) was responsible for creating an app to organize evidence and managing the access-rights. At the end of this project, we transferred that knowledge to an internal Dutch Police-team.

What I did:

- Worked together with a team of highly skilled professionals;
- Retrieved and prioritized the system requirements and user requirements;
- Setup a completely new Angular 9-project (later converted to 11);
- In Swagger (OpenApi 3), I designed a REST API and optimized it for a snappy frontend experience. This also included a partial response (a kind of simplified GraphQL);
- Directed the conversation with the internal Dutch Police-team to implement the API-design in the backend;
- Helped the internal team to create actions from that conversation;
- Minified the build-time in the CI/CD pipeline from 18 minutes to just 6, by applying caching and other optimizations;
- Setup rolling updates in the pipeline so there won't be any downtime.

The result:

An API-design that's been built from the system and user requirements, which serves as a single source of truth for the back- and frontend. These API-responses are tested thoroughly, which greatly improves the maintainability for this very complex and big application.

Next to that, I delivered an Angular app that was made according to the system requirements and usertests. It was created with the greatest care and while keeping in mind that I wouldn't be there to maintain it - which meant very typesafe and loads of quality tests to ensure that the functionality wouldn't regress (break).

Stack (not ordered): JavaScript, NPM, yarn, Source Tree, CI / CD, Prettier, HTML(5), (S)CSS(3), Swagger, OpenApi 3, Docker, Kubernetes, Cloud Foundry, Python, Jasmine, Protractor, Angular, TypeScript, TSLint (and converted that to ESLint), RxJS, Node, Express

Used specializations and methodologies: Agile / Scrum, flow design, Git, Accessibility, UX design, usertests, unit & component testing

Jan 2019 - Apr 2020

Software Engineer

Lidl (via App4mation), Utrecht

At App4mation I worked on several Angular apps.

My task:

- Create enterprise applications for a.o. ING, Heraeus, Lidl (Schwarz IT) and Finanz Informatik;
- Implement designs in Angular (ING, Heraeus and Lidl);
- Direct contact with the clients (ING and Lidl);
- Refactor big technical debts (ING);
- Focus on speed and refactoring so the application would run faster (ING);
- Design and create REST API's (Heraeus, ING and Lidl);
- UX-research and creating a design with/for the customers (Heraeus, ING and Lidl);
- I realized I really like helping juniors and impart knowledge (zie Joel's recommendation ↓↓↓).

Stack (not ordered):

Agile / Scrum, Angular, UX design, Git, GitLab, JavaScript, Source Tree, HTML(5), (S)CSS(3), unit & component testing

Nov 2017 - Jan 2019

Frontend developer

PANGAEA Digital Agency, Den Haag

For big clients in catering industries and retail we made enterprise websites and maintained those. These websites ran on a CMS that PANGAEA developed itself, which was tailored for these kinds of clients.

What I did:

- Create enterprise websites for big clients like DEEN, Hotel van Oranje, Flemings, Hotel Arena, Duif, GreenSand, Hotel Amsterdam and Bernard Sanders;
- Brought structure to existing code using scss, BEM and React;
- Setup the structure of projects and figure out the best way of approaching them;
- In GitHub and GitLab I setup CI/CD and investigate GitFlow, GitHub Flow and other forms of collaborating;
- Setup big projects and maintained them;
- Did a lot of knowledge-sharing via internal articles.

Stack (not ordered): JavaScript, jQuery, React, XML, NPM, Source Tree, HTML(5), (S)CSS(3), Razor, Jira, Confluence, GitLab, GitHub, Photoshop, Balsamiq

Used specializations and methodologies: UX design, Accessibility, CI / CD, Agile / Scrum, Git, Git flow, Github flow

Oct 2011 - Nov 2021

Software Engineer & owner

Den Braber Webdesign, Veenendaal

With Den Braber Webdesign, I focused on strong client relationships and building a wide variety of applications and websites. When I transitioned to full-time freelance work, the workload became too much to continue running Den Braber Webdesign alongside, so I decided to discontinue it.

What I did:

- Built many websites using a variety of techniques,
- Strong focus on client needs and user experience,
- Prioritized accessibility and support for diverse demographic groups,
- Performance-focused development — many sites scored above 80 on Google PageSpeed; for instance, denbraberwebdesign.nl scores 98 and lexwart.nl also performs very well.

Stack (not ordered): JavaScript, jQuery, PHP, Accessibility, Prettier, HTML(5), (S)CSS(3), WordPress, FTP, GitLab, GitHub, Python, Django, Angular

Used specializations and methodologies: Git, SEO, SEA, performance optimization, CI/CD, Flow design, design, UX design, customer communication

Feb 2015 - Jul 2016

Frontend developer

ABF Research, Delft

At ABF I was responsible for developing websites, mostly for municipalities.

What I did/learned:

- Created several websites for municipalities for reviewing data, a.o. Hardinxveld-Giessendam, Rotterdam and Delft;
- Used CSS, HTML and Javascript to make the designs as efficiently as possible;
- Quickly creating sketches using Balsamiq and aligning with colleagues;
- Learned that I love creating things instead of just designing them, mostly because of Hing and his expertise (see his recommendation on LinkedIn).

Stack (not ordered):

JavaScript, jQuery, JSON, XML, Accessibility, UX-design, Prettier, ESLint, HTML(5), (S)CSS(3), Flow design, Razor

Languages

Dutch	● ● ● ● ●
English	● ● ● ● ●
German	● ● ● ● ●

Courses

Nov 2021	●	Learn NgRx from the makers of NgRx (day 2) <i>Pluralsight</i> handle errors · fx (mergeMap: delete ° exhaust-: non-param queries ° switch-: param queries ° concat-: update/create) · Entity w/ adapter · meta-reducers · actions: events, not cmds ° be specific ° no subtyping
Oct 2021	●	Learn NgRx from the makers of NgRx (day 1) <i>Pluralsight</i> how things work behind the scenes · reducers · actions · using typesafe effects · action creators · typesafe and memoized selectors · barrel files · using Immer for safe reducers
Aug 2021	●	Ultimate Redux <i>Code with Mosh</i> composing and piping · Immer · how it really works · inspector and tracing · redux toolkit · slicers · moization · middleware · consuming api's · testing (solitary/social, clean tests)
Jun 2022	●	Mastering React <i>Code with Mosh</i> routing · working with forms and validation · connecting to backend · auth · higher order components · useState hook · useEffect hook · custom hooks · context in class and functional components
Oct 2021	●	NgRx <i>Pluralsight</i> redux pattern advantages · working with the store · dispatching vanilla actions · dispatching with TypeSafe action creators · using effects · CRUD operations · debugging tools · working with shell and presentational components (OnPush strategy)
May 2021	●	StencilJS <i>Udemy</i> (dynamic) styling · hostdata · two-way binding · lifecycle hooks · emitting and listening to events · debugging · deploy to npm/basic/Angular/Vue/React ■ Native webcomponents ☒ shadow DOM · (dynamic) styling · emit/listen to events
Apr 2021	●	Docker <i>Udemy</i> commands · advanced run features · creating new images · (converting to) docker-compose · registry · storage · networking · orchestration with Swarm and Kubernetes

Apr 2021

●

Git

Code with Mosh

git background · advanced history · bisecting · restoring · tagging · difftools · aliases · fast-forward and three-way merge · advanced rebasing: undo / recover / amend / drop / reorder / squash / fixup / split commits

Feb 2021

●

(Node) NestJS

Udemy

structuring your code · CRUD (PostgreSQL) · Data Transfer Objects (!) · (Custom) pipes · Validating input · Data persistence · Repository pattern · (Custom) error handling · JWT strategy (auth²) · deploying to AWS · advanced testing

Sep 2021

●

Node.js

Code with Mosh

conv. default callbacks to promises · custom middleware · Express&Mongo api · (async) validators · api persistence · modelling relationships · JWT auth² protecting routes · advanced error handling · advanced integration & unit testing w/ Jest

Oct 2020

●

Python

Code with Mosh

xargs · xxargs · list and dict comprehensions · stacks · queues · cost of raising exceptions · abstract base classes · polymorphism · working with zip, json, csv · pipenv · docstrings · Pydoc · sending text messages · NumPy

Sep 2020

●

Python

Code with Mosh

receiving input · type conversion · arithmetic operations · Math functions · (nested) for-loops · tuples · unpacking · exceptions · classes · constructors · inheritance · modules · packages · Pypi and Pip

Apr 2020

●

Angular testing

Pluralsight

testing a pipe, service, component · mocking to isolate code · TestBed · mocking injected service · deep integration testing · mocking HTTP · emitting children events · mocking routerlink · async code · code coverage · spies

Apr 2020

●

Java

Pluralsight

data types · abstract interfaces · inheritance · constructors · arrays and objects · creating abstract relationships with interfaces · nested types · anonymous classes

Apr 2020

●

Java

Code with Mosh

exceptions · generics · collections · lambda expressions · functional interfaces · streams · concurrency & multi-threading · executive framework

Apr 2020

●

Java

Code with Mosh

memory allocation · abstraction · refactoring to OOP · polymorphism · deep inheritance · best way of coupling · injection · why and how to use OOP · why and how to use interfaces · multiple projects

Apr 2020

● **Java**

Code with Mosh

primitive types · reference types · multi-dimensional arrays · arithmetic expressions · casting · reading and validating input · ternary operator · loops · clean coding · debugging runtime errors · packaging

Apr 2020

● **RxJS**

Pluralsight

operators · marble diagrams · create your own operators · Subjects · hot & cold Observables · multicast operators instead of Subjects · multicast specialized operators · execution schedulers · testing with TestScheduler and helpers

Mar 2020

● **Angular for enterprise**

Code with Mosh

advanced Firebase · Advanced Ionic2 · integration with ASP.NET MVC/Core · Angular change detection · Advanced Redux · Advanced unit, integration and e2e testing

Feb 2020

● **Design Patterns in Java & TypeScript**

Code with Mosh

coupling · encapsulation · abstraction · inheritance · polymorphism · UML ■ The patterns → Memento · State · Iterator · Strategy · Template · Command · Observer · Mediator · Chain of responsibility · Visitor

Jan 2020

● **Angular**

Code with Mosh

template-driven forms · reactive forms · http-services · routing · authentication and authorization · deploying to Firebase, Heroku · Animations · Material · Redux · Unit, integration and e2e testing (30 uur)

Nov 2019

● **ES6**

HackerYou partner BosType Inc

arrow functions · destructuring objects and arrays · promises · linting · npm · classes · generators · Proxies · Sets & WeakSets · Map & WeakMap · async & await · Object methods

Aug 2019

● **Certified Application Developer**

ServiceNow

create complex applications · design application tables · create dynamic forms · controlling access and security · create api's · integrate workflows and business logic

Jun 2019

● **Certified System Administrator**

ServiceNow

manage simple applications · read from the database · learning to secure simple things · make use of api's · reading and acting on business logic

Education

2012 - 2018

● **HBO CMD**

Haagse Hogeschool, Den Haag

Interests

Pingpong, reading, fitness at home, enlarging my knowledge (also see my courses), having a great time with friends, my wife, colleagues or family.

What my colleagues said about me

See [linkedin.com/in/aartdenbraber](https://www.linkedin.com/in/aartdenbraber) for the people behind these recommendations. I'm very proud that they were so happy working with me that they were willing to write something!

Bart Illy, product owner:

As a Product Owner, I have worked closely with Aart. Aart is a perfectionist and socially involved. As a valued team player and Frontend developer, he has made an important contribution to the development of the online banking environment. Among other things, he has fully used his knowledge of Angular and React. Together with his UX and Test knowledge, I would like to recommend Aart wholeheartedly.

Anton Zhirkov, colleague:

I worked for a year with Aart in close collaboration and it was a fantastic time.

Exceptional hard and soft skills of Aart were crucial for the fast development of several frontend applications based on Angular and React, backend using NodeJS, establishing CI/CD and deploying on AWS.

I would also highlight his hardworking approach, striving for excellence in code quality, testing and UX. Definitely recommend him to any company that is looking for excellence and value.

Thijs Knippers, colleague:

I've come to know Aart as the frontend developer for the Pandora project at the Police where he was the key developer in defining a new API for an existing application. He was able to properly translate the wishes from key users through ux testing and then specifying those requirements into a detailed API (fully documented in Swagger) and providing a frontend UI for that very same API as well, with its own set of automated tests. If that doesn't tell you how multi-talented he is, I don't know what will :). Moreover, Aart is always willing to learn new skills and at the same time teach team members from his experiences with a patience and smile you'll only find in the best of teachers. He's willing to discuss new ideas, give and receive feedback and readjust to new information, keeping the team at their best.

Ravi Selker, colleague:

Aart is a great software developer. He really emphasises the importance of writing comprehensible and well-tested code, and is able to use state of the art techniques to accomplish this. I have had the pleasure to work with Aart in a project where he was the lead front-end developer, and I was a back-end developer that wanted to learn more about front-end development. He happily took on the additional role of front-end mentor, and was always happy to explain things to me and help me gain more hands-on experience through pair-programming sessions. In sum, Aart is a great asset in every project where you need a social, enthusiastic developer that can both design and create a future-proof and maintainable application.

Raphael Rodriguez, manager:

Aart has demonstrated great ability in dealing with a highly complex technical project in a challenging setting. He was part of a multi-country development team and he played an essential role in having a successful project with a happy customer.

He saw great technical challenges as great opportunities rather than negative difficulties.

We are very grateful for the knowledge and assistance that Aart brought into our project.

And 18 more on [linkedin.com/in/aartdenbraber](https://www.linkedin.com/in/aartdenbraber)